[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Genetic Algorithms Applications and more Advanced Techniques

University of Essex
School of Computer Science and Electronic Engineering

Today's learning objective

- Describe how Genetic Algorithms can be used to solve **unconstrained function optimisation** and **unconstrained combinatorial optimisation** problems.
- Develop appropriate **fitness** functions for various problems.
- Describe **crossover** and **mutation** operators for **real-valued Genetic Algorithms** (GAs) and discuss possible ways to implement them.
- Describe **rank selection**
- Summarize different cases of mapping **objective functions** into **fitness functions**.
- Discuss **fitness functions** in terms of **constrained optimization** problems

Constrained vs Unconstrained Optimisation

- **Unconstrained optimisation** means that we are looking for optimal solutions in a search space where *every point we can visit is a valid solution*.
- **Constrained optimisation** means that we are looking for optimal solutions in a search space where there are *places that represent invalid or unacceptable solutions* (constraints). So, we must avoid such places.

GA for Unconstrained Optimisation

- GAs can be easily used for:
  - **Unconstrained <u>function</u> optimisation**, i.e., to find the values for the parameters that minimise or maximise a given objective function $U(x)$
  - **Unconstrained <u>combinatorial</u> optimisation**, i.e., to find the best combination of choices (for example integers) that optimise the function $U(x)$

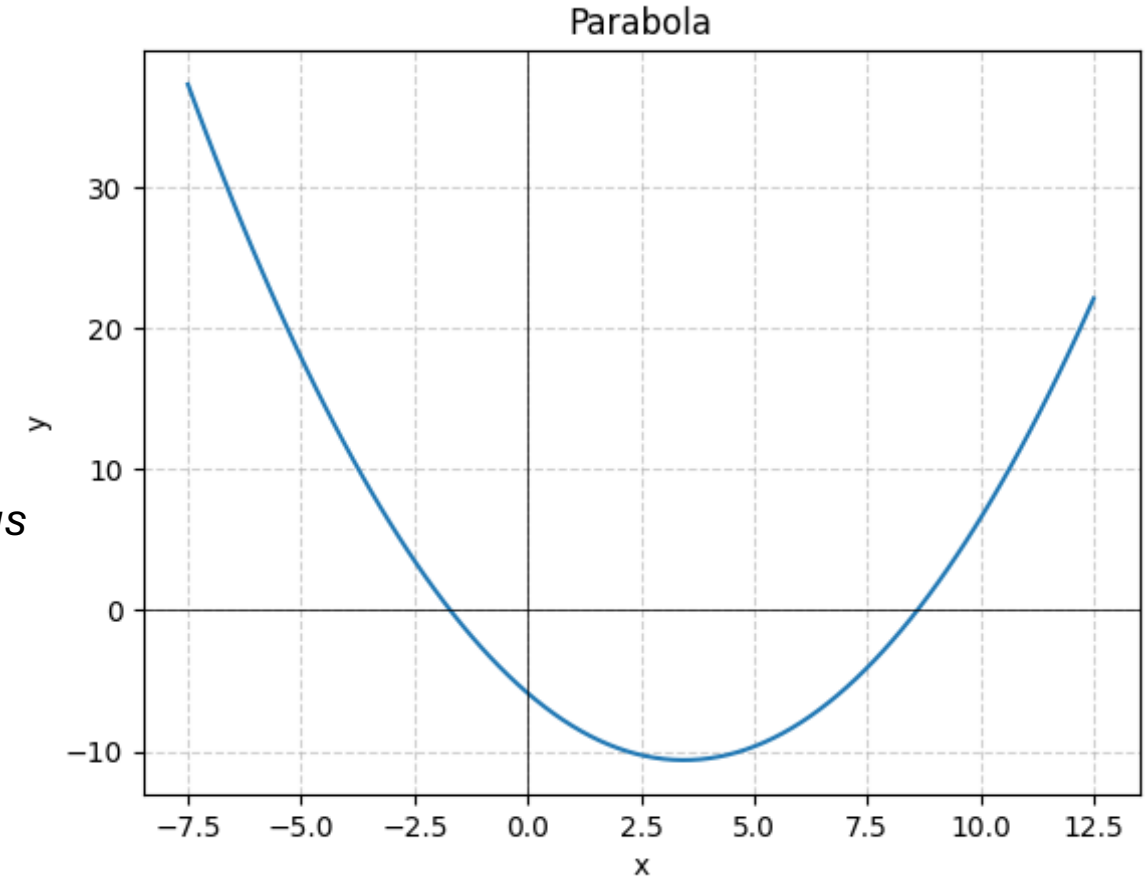
The difference is only in the nature of the variables x: *continuous* vs. *discrete*.

Example: unconstrained function optimisation

- Find the minimum* of this parabola: $y = 0.4\,x^2 - 2.76\,x - 5.864$

*Meaning: find the *x* value where the *y* value is lowest

NOTE: x is continuous (real/float)



Example: unconstrained <u>combinatorial</u> optimisation

- The **bin packing** problem!
- It is **combinatorial** because solutions are represented by discrete quantities (permutations/lists of integers)
- It is **unconstrained** because we have as *many bins as needed*, so *any permutation is a valid assignment* of items to bins.
- Note, it would be <u>*constrained*</u> optimisation if:
  - We have a *limited number of bins*. So, permutations that run out of bins are <u>invalid</u>
  - If we used as a representation *any list of integers* (not guaranteed to be a permutation), lists where some items are *binned twice* or are *never binned* are <u>invalid</u>

Example 1: Neuron Evolution

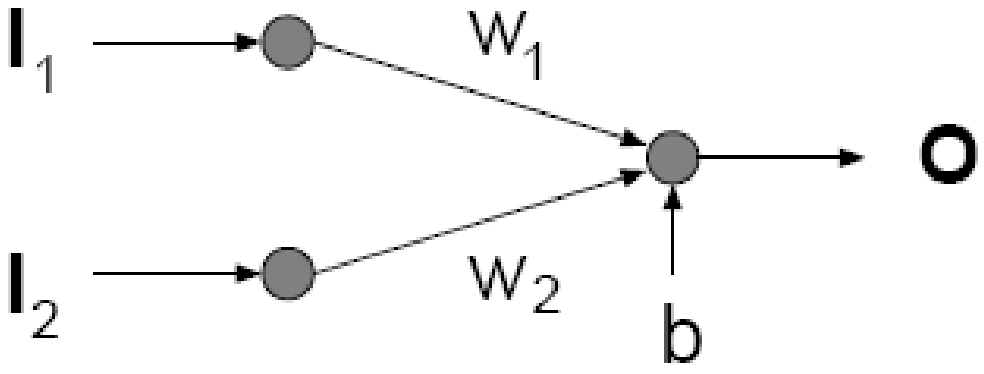
- We are given this artificial neuron:
- First it computes its net input:

$$net = w_1 \cdot I_1 + w_2 \cdot I_2 + b$$

- Then it computes its output:

O(net) = 1 if $net > 0$

O(net) = 0 otherwise




```
def output(I1,I2,w1,w2,b):
    net = w1*I1 + w2*I2 + b
    if net > 0:
        return 1
    else:
        return 0
```

Example 1: Problem

- Find weights (w_1 and w_2) and bias (b) so that the neuron implements the **Boolean OR** function

FITNESS FUNCTION?

- Our objective function is to minimise the *error*:

$$U(w_1, w_2, b) = \boxed{\text{abs}(O_1 - T_1)} + \text{abs}(O_2 - T_2) + \text{abs}(O_3 - T_3) + \text{abs}(O_4 - T_4)$$

I_1	I_2	T (I_1 OR I_2)
0	0	0 T1
0	1	1 T2
1	0	1 T3
1	1	1 T4

where:

- each term represents a <u>fitness case</u> (e.g., a row of the truth table),
- T_1, T_2, ... is the **desired output** of the neuron for a fitness case
- O_1, O_2,... is the corresponding **actual output** on the neuron

```
def output(I1,I2,w1,w2,b):
    net = w1*I1 + w2*I2 + b
    if net > 0:
        return 1
    else:
        return 0
```

Example 1: Solution approach

- Let us use a binary GA to solve the problem
- **Parameters of the GA**:
  - **One-point crossover** with p_c = 0.7
  - No mutation
  - **Tournament** selection ($N = 2$)
  - **Chromosomes** = strings $\boldsymbol{e_1 e_2 e_3}$ with e_i in the set {-1, 0,1}
  - **Search space size** = 3^3 = 27 (# different networks)
  - **Fitness**: $U(w_1, w_2, b) = \textit{number of errors}$ (worst=4, best=0), so…

$$\boldsymbol{f} = error_{max} - error = \mathbf{4} - \boldsymbol{U}(\boldsymbol{w_1}, \boldsymbol{w_2}, \boldsymbol{b})$$

  - Number of individuals = 4
  - Number of Generations = 3

Example 1: A few generations by hand (1)

Pop 0	Fitness	Pop 1	Fitness
-1-1 0	1		
-1 0 1	1		
0 0 1	3		
1 1-1	2		
	1.75		

average

TOURNAMENT

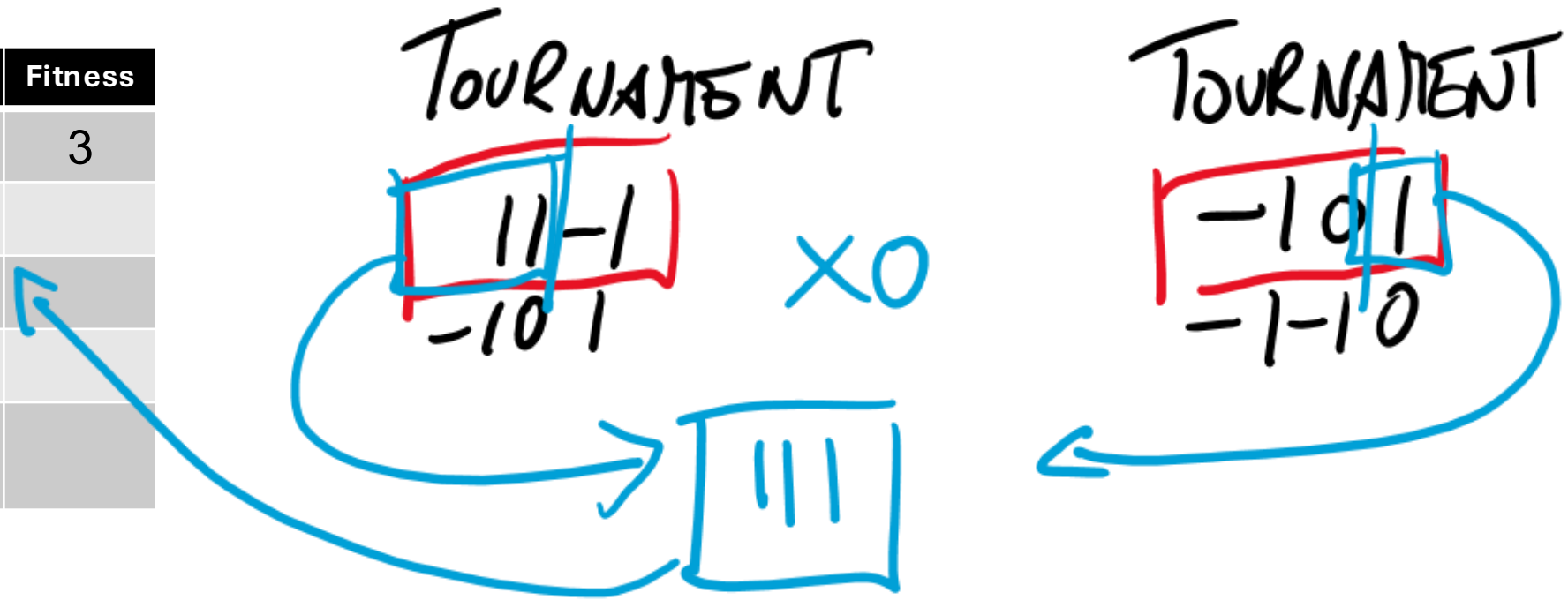
-1 0 1

$\boxed{0\ 0\ 1}$

CLONING

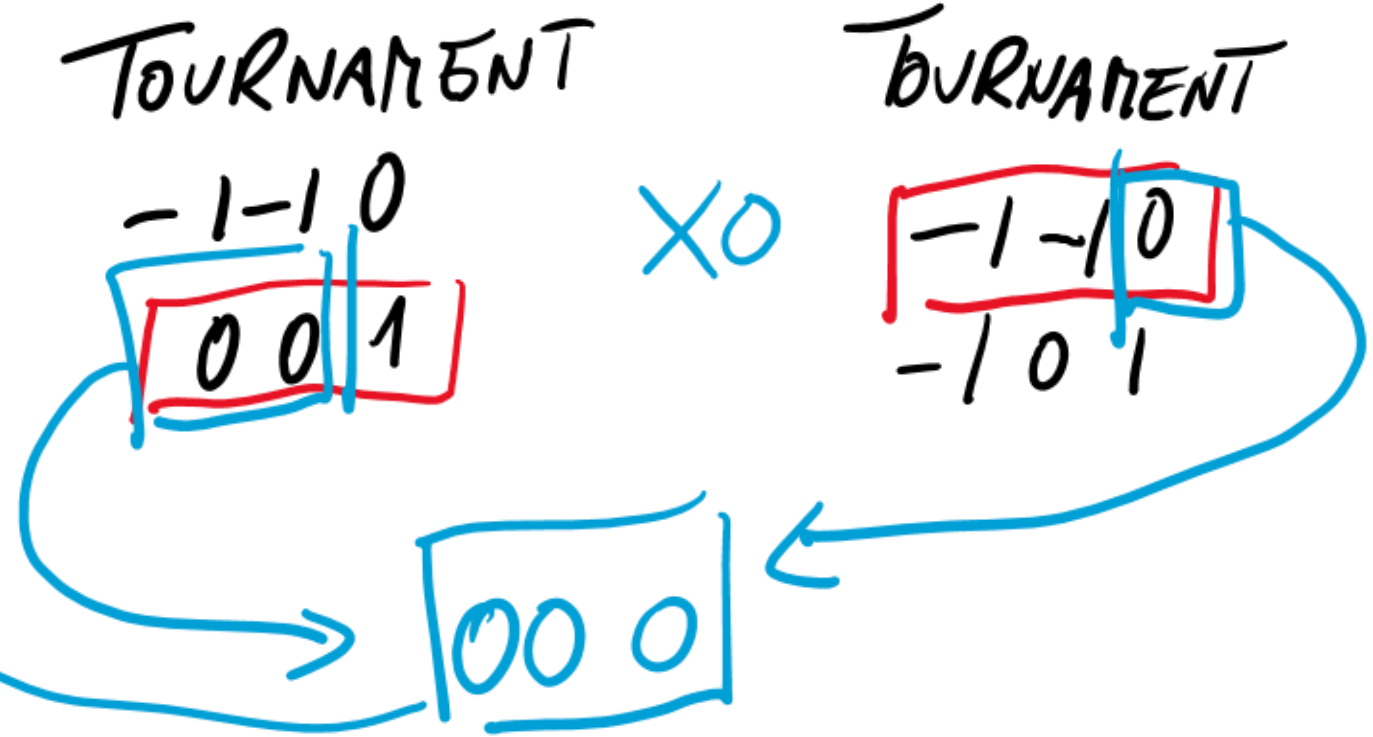
Example 1: A few generations by hand (2)

Pop 0	Fitness	Pop 1	Fitness
-1-1 0	1	001	3
-1 0 1	1		
0 0 1	3		
1 1-1	2		
	1.75		



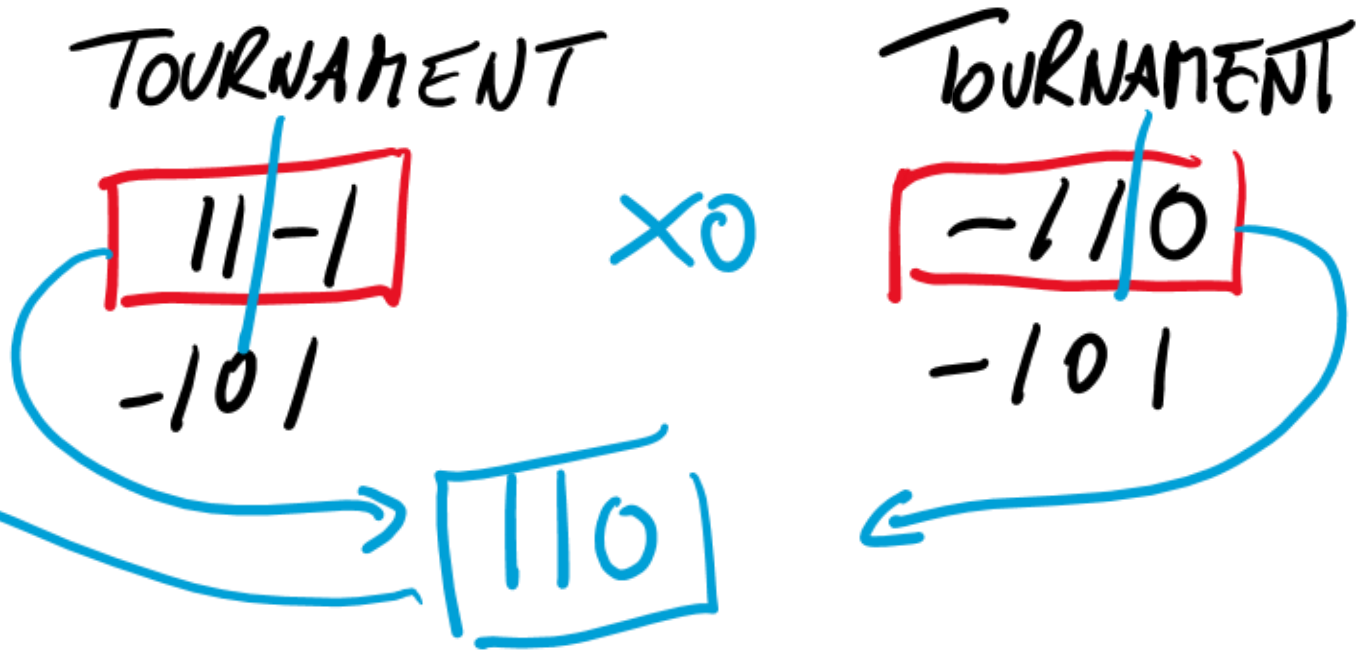
Example 1: A few generations by hand (3)

Pop 0	Fitness	Pop 1	Fitness
-1-1 0	1	001	3
-1 0 1	1	111	3
0 0 1	3		
1 1-1	2		
	1.75		



Example 1: A few generations by hand (4)

Pop 0	Fitness	Pop 1	Fitness
-1-1 0	1	001	3
-1 0 1	1	111	3
0 0 1	3	000	1
1 1-1	2		
	1.75		



Example 1: A few generations by hand (5)

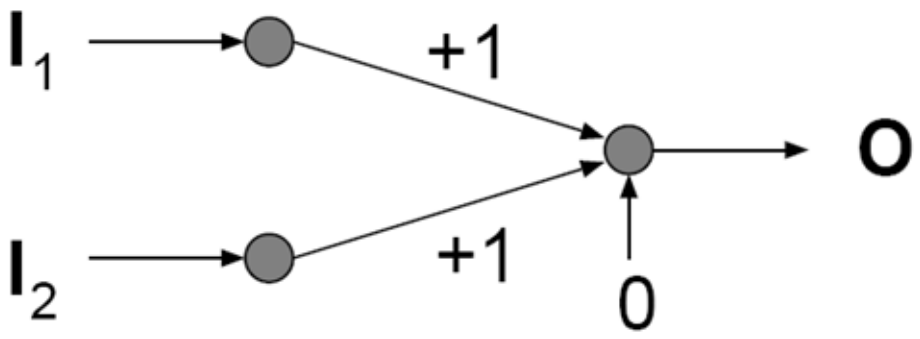
Pop 0	Fitness	Pop 1	Fitness	Pop 2	Fitness
-1-1 0	1	0 0 1	3	1 1 0	4
-1 0 1	1	1 1 1	3	1 0 1	3
0 0 1	3	0 0 0	1	1 1 0	4
1 1-1	2	1 1 0	4	1 0 1	3
	1.75		2.75		3.5

average

average

Example 1: A few generations by hand (6)

- The best string is **110** which corresponds to a neuron that implements exactly the OR function:



- **Solutions sampled**: 4 (individuals) x 3 (generations) = 12
- **Fraction of search space explored**: 12 / 27 = 44%

Example 2: Sphere Model

- We want to **minimise** the function

$$U(x_1, x_2, x_3) = x_1 * x_1 + x_2 * x_2 + x_3 * x_3$$

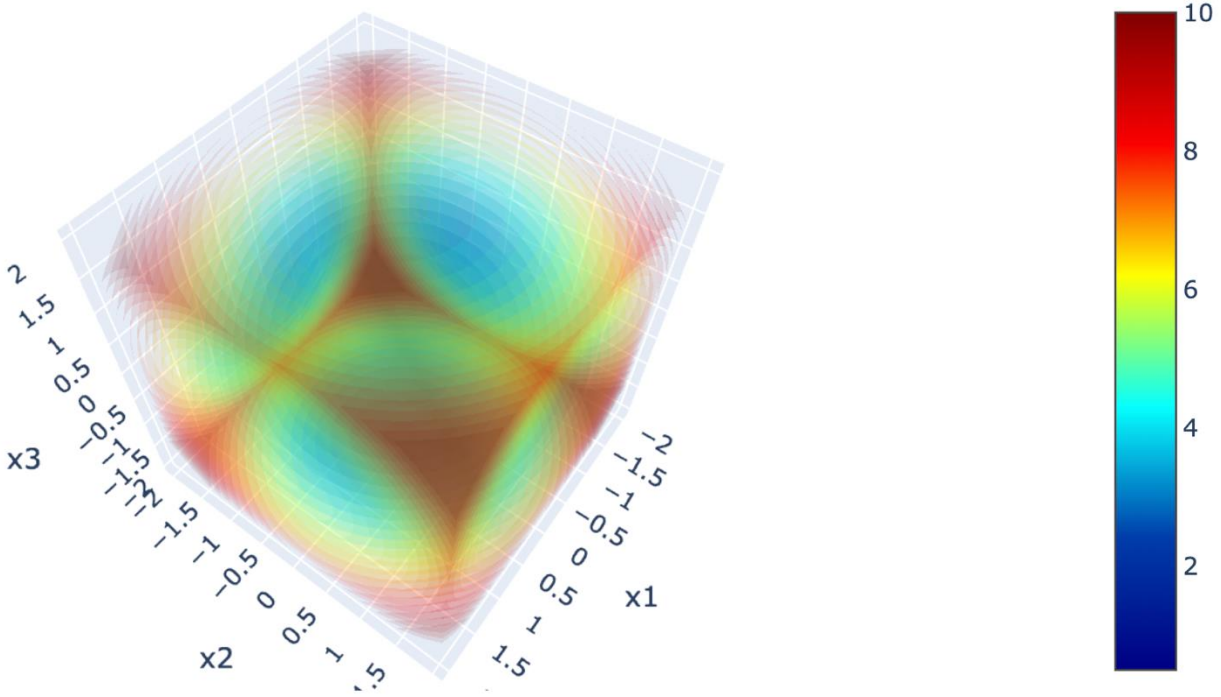
with x_i real values in the **range [-10, 10]**.

- The known minimum is (0,0,0)

- This function can be visualised only the equivalent of a heatmap or by reducing its dimensionality (e.g., by “slicing” or projecting its “3-D shadow”).

Example 2: Sphere Model

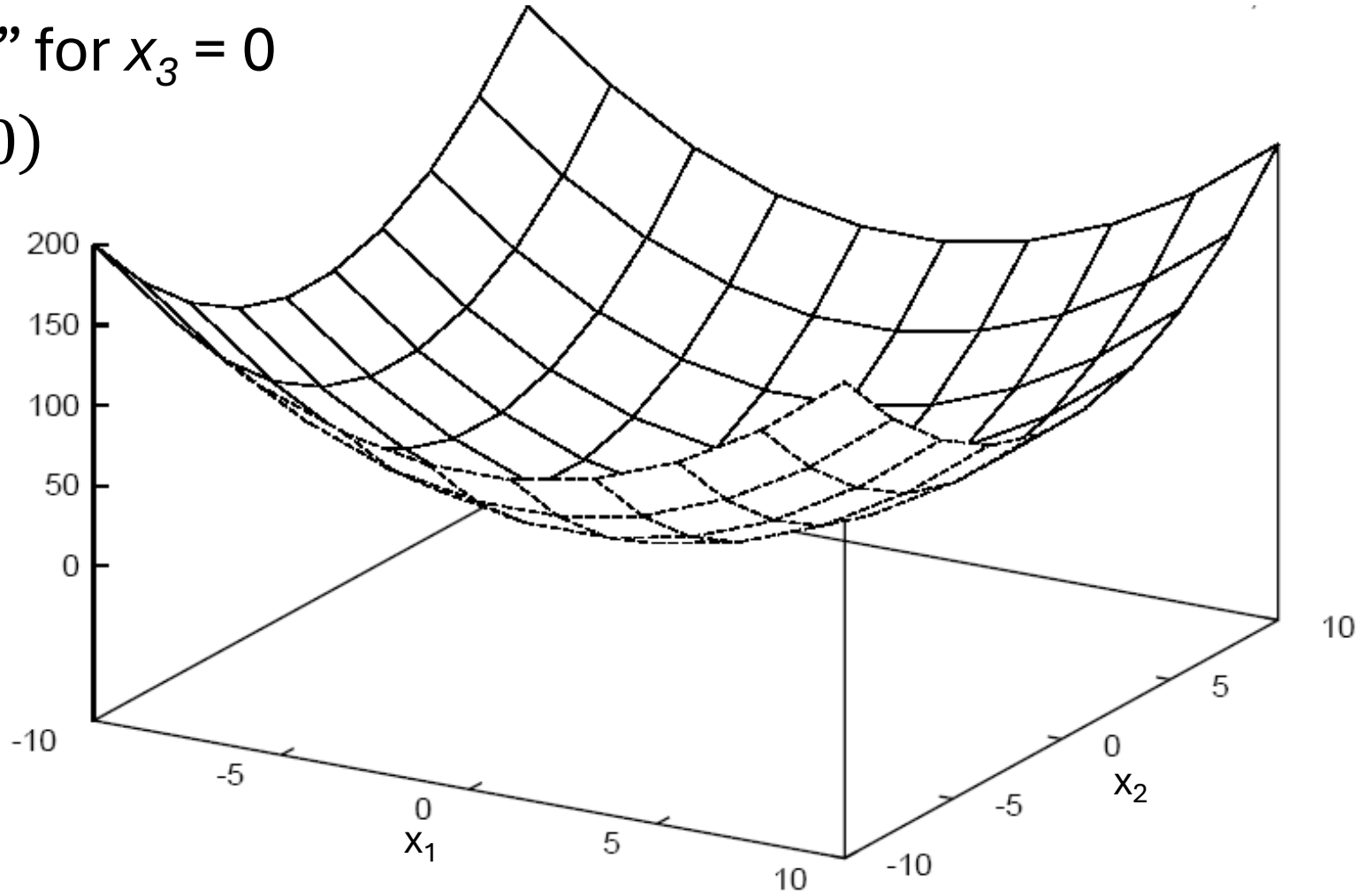
- Here it is as a 3-D heat map



Example 2: Sphere Model

- Here a plot of is its "slice" for $x_3 = 0$

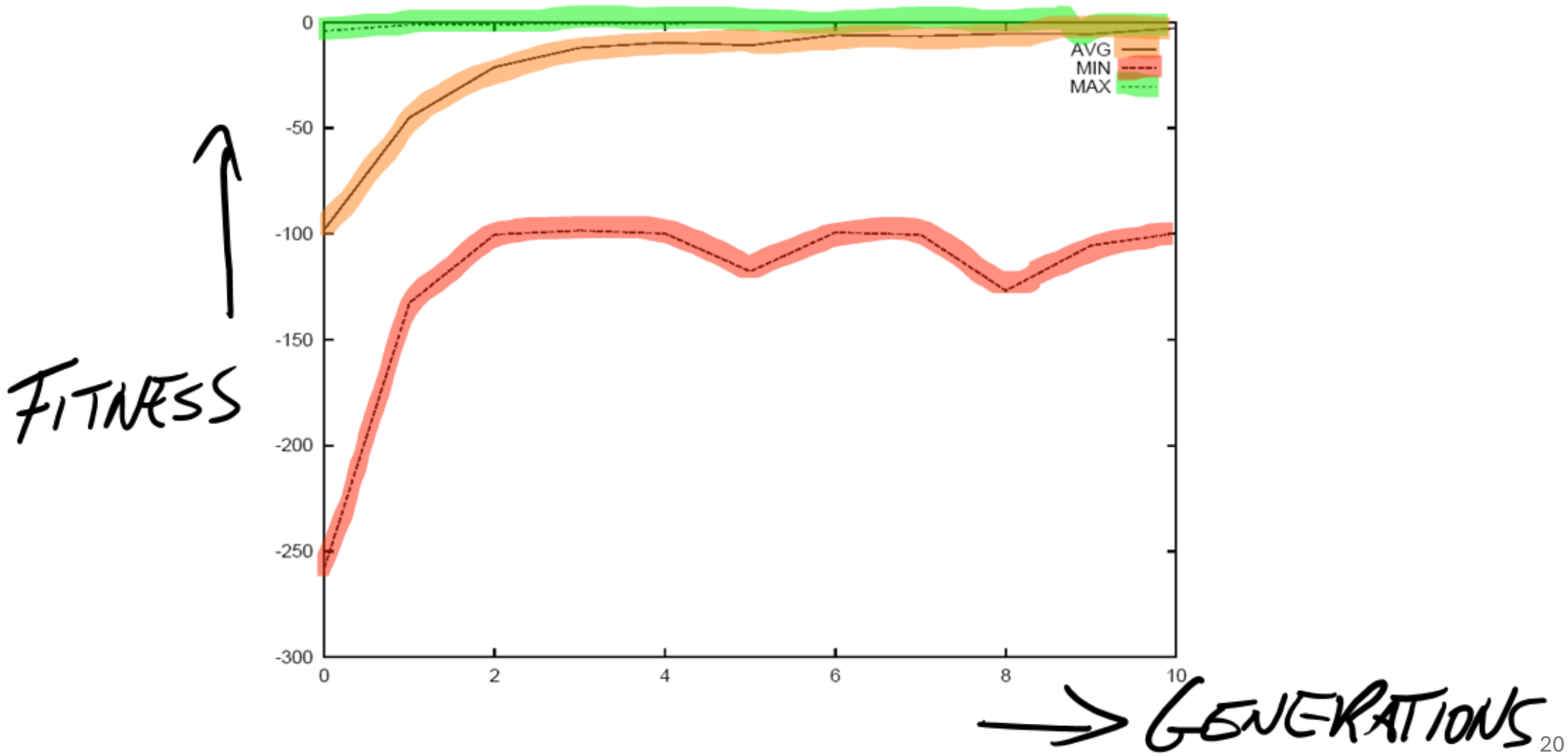
$$U_{slice}(x_1, x_2) = U(x_1, x_2, 0)$$



Example 2: Solution approach

- Let us use a binary GA to solve the problem
- **Parameters of the GA**:
  - **One-point crossover** with $p_c = 0.7$
  - **Mutation** with $p_m = 0.01$
  - **Tournament** selection (N=2)
  - Chromosomes with **3 × 10 = 30 bits**
    (Resolution = interval / #possible values = 20 / 2^{10} = 20 / 1024 = 0.0196)
  - **Size of search space**: 2^{30}=1,073,741,824
  - **Fitness** function: $f = -U(x_1, x_2, x_3)$
  - Number of individuals = 200
  - Number of Generations = 50

Example 2: Fitness vs. generation plot



Example 2: Solution

- Best solution after 50 generations:

  0000000001 1111111110 0000000001

  (LSB to MSB), that is x1 = x3 = 0.009777 and x2 = -0.009777, which corresponds to **U = 0.000287**.

- *Why don't we get* $\boldsymbol{x_1 = x_2 = x_3 = 0.0}$ *(and* ***U=0.0****)?*

  Hint: ***0.009777 = Resolution / 2***

- Solutions sampled: 200 x 50 = 10,000
- Fraction of search space explored: **0.00093%**

Example 3: π (3.1415…)

- Find the best *rational approximation* of π using integers between 0 and 1023

i.e., **find x_1 and x_2 such that x_1 / x_2 is as close as possible to π**

- This can be recast as:

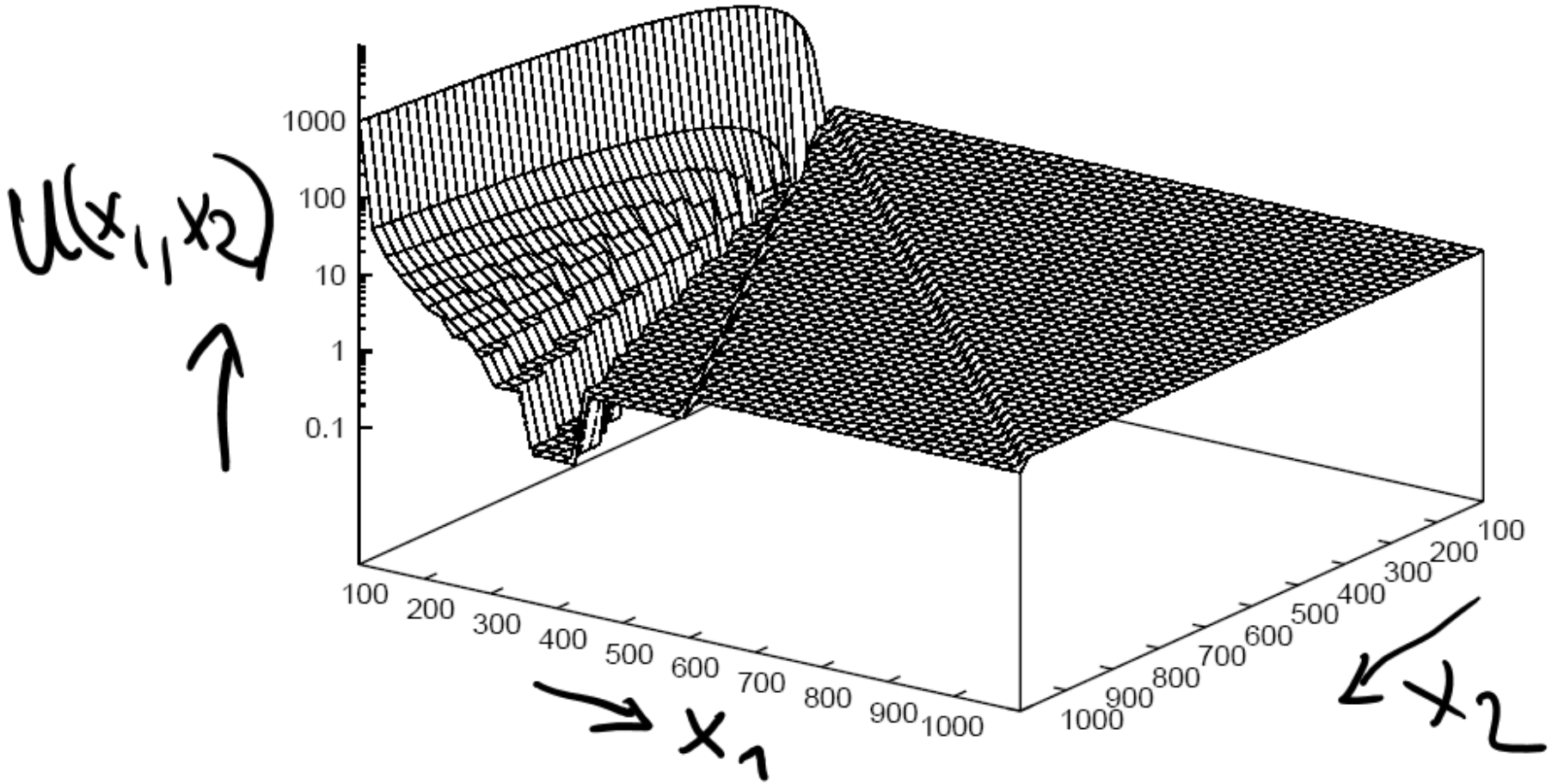
minimise the function

$$U(x_1, x_2) = abs\left(\frac{x_1}{x_2} - \pi\right)$$

with x_1 and x_2 in the set {0,1,…,1023}

- A good solution is 22/7, but we want the best!

Example 3: Plot of $U(x_1, x_2)$

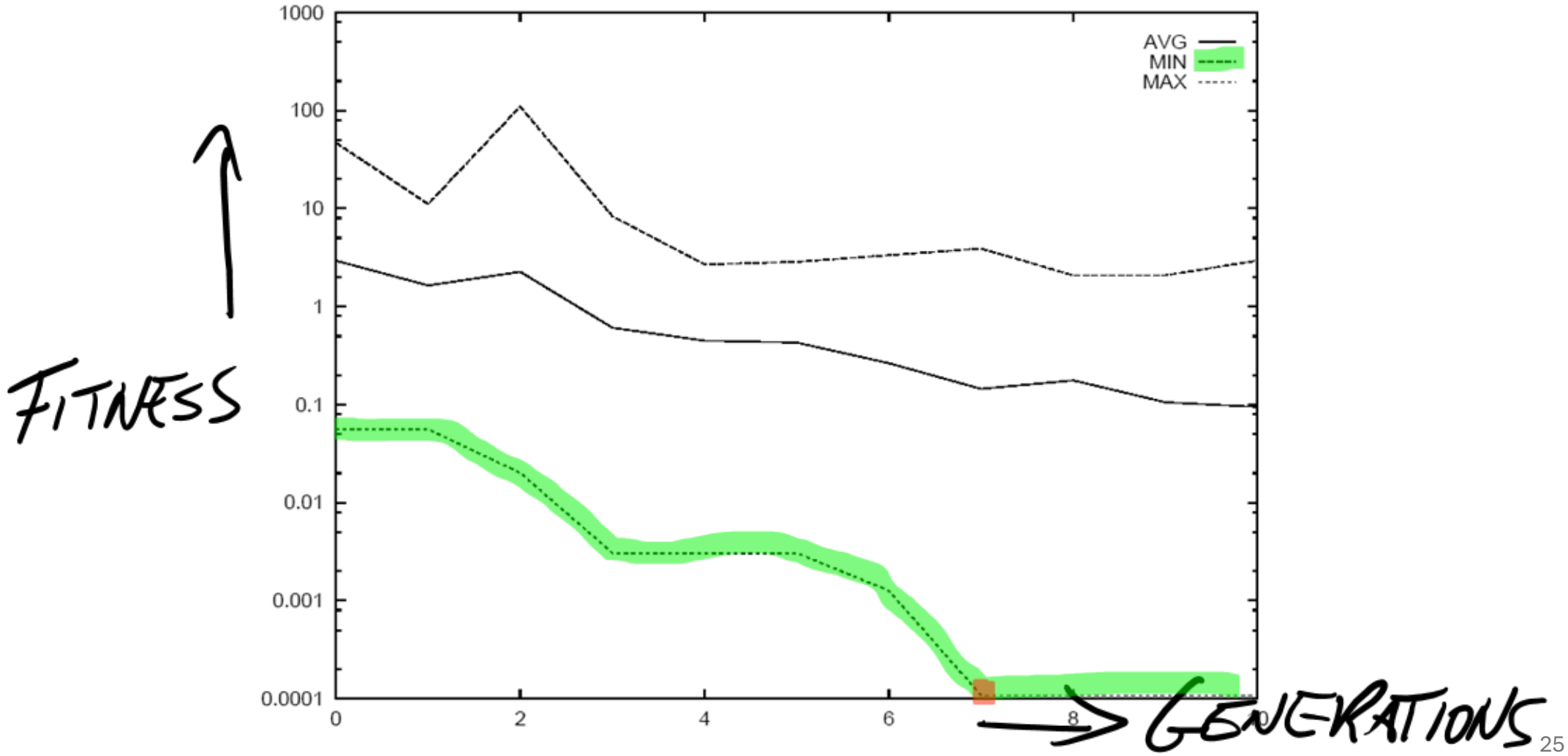


- *Brute force search* (test all possible pairs) finds **355/113** (U<0.00001).

Example 3: Solution approach

- Let us solve the problem using a binary GA
- **Parameters of the GA**:
  - **One-point crossover** with $p_c = 0.7$
  - **Mutation** with $p_m = 0.01$
  - **Tournament** selection (N=3)
  - **Chromosomes** with $2 \times 10 = 20$ bits
  - Search space size: $2^{20} = 1{,}048{,}576$
  - **Fitness** function = $-U(x_1, x_2)$
  - Number of individuals = 100
  - Number of generations = 50

Example 3: fitness vs. generations plot



Example 3: Solution

- Best solution after 50 generations:
  1110000011 0110111100
  (LSB to MSB), that is
  x1 = 776 and x2 = 247
  which corresponds to U = 0.000108.
- Solutions sampled: 5,000
- Fraction of search space explored: 0.48%

Real-valued GAs

- In real-valued GAs each gene is represented by a **floating-point number**.

17.525	0.001	27.273	−23.500

- These GAs are particularly suited for problems where the problem variables are naturally represented as real numbers within pre-specified intervals.
- They have no problems zooming in onto solutions **without the resolution barriers of binary GAs**.

Real-valued GAs

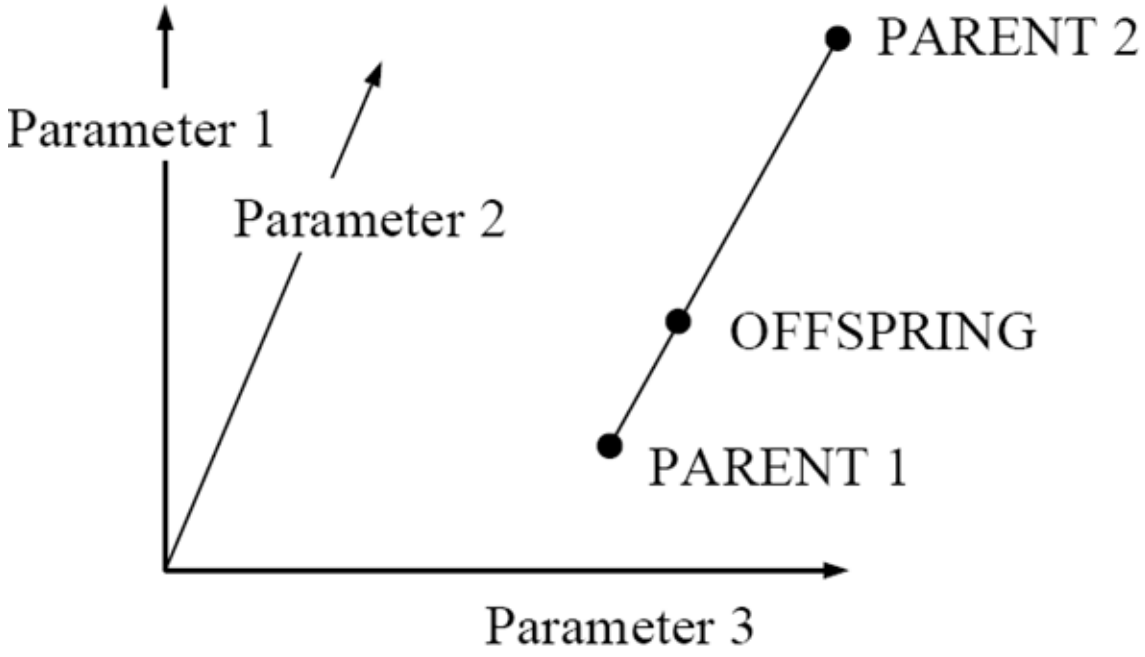
- It is possible to use standard crossovers on vectors of real-valued parameters, e.g.,

Parent a	**0.3**	**0.2**	0.5	0.7	1.0
Parent b	0.5	0.3	**0.7**	**0.1**	**0.1**
Offspring ab	**0.3**	**0.2**	**0.7**	**0.1**	**0.1**

- but this may lead to a **poor search** of the **parameter space** because crossover cannot create any new values.

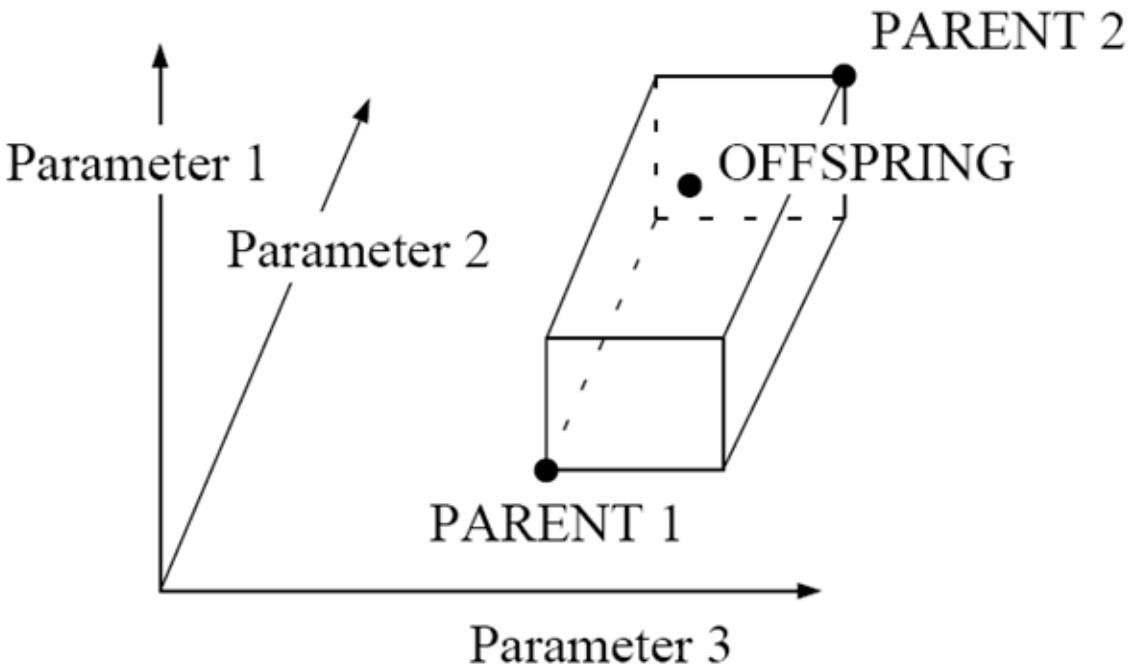
Crossover

- In real-valued GAs crossover is often seen as an **interpolation** process in a multi-dimensional space.



Crossover

- Alternatively, crossover can be seen as the exploration of a **multi-dimensional** (**hyper-**) **box** defined by the parents.



Implementation

$$\text{Offspring}[1] = \text{Parent1}[1] * R_1 + \text{Parent2}[1] * (1 - R_1)$$

$$\vdots$$

$$\text{Offspring}[N] = \text{Parent1}[N] * R_N + \text{Parent2}[N] * (1 - R_N)$$

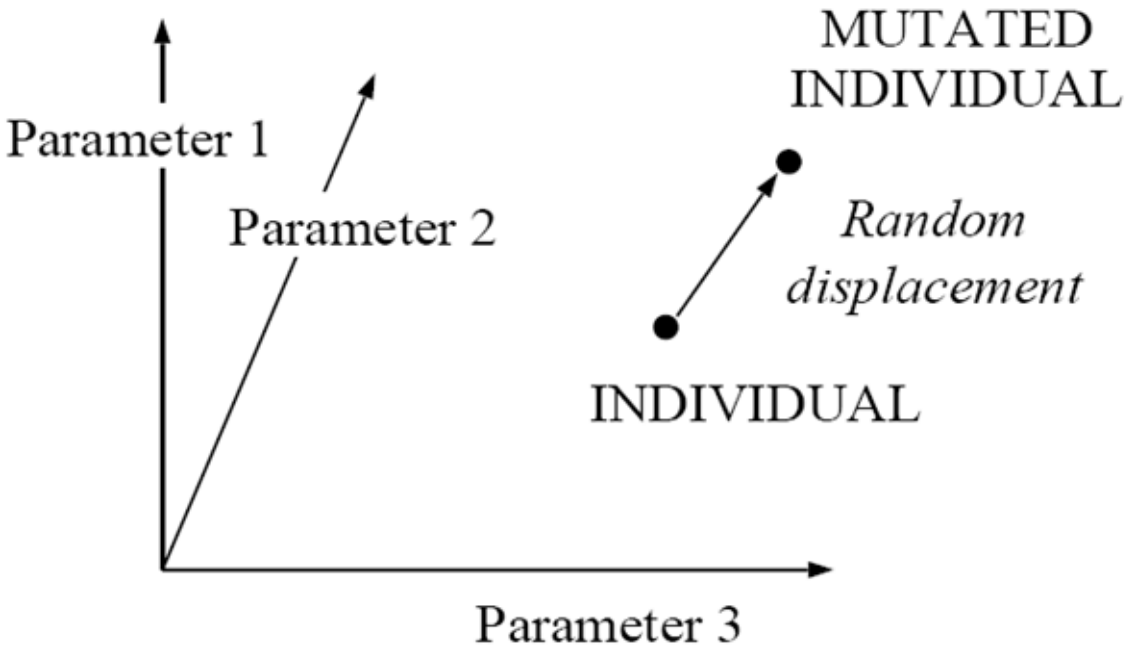
where R_1, ..., R_N are random numbers <u>between 0 and 1</u>

If $R_1 = \ldots = R_N$ = **same** random number → **Interpolation** Crossover

If R_1, R_2, ...R_N = **different** random numbers → **Box** Crossover

Mutation

- Mutation is often seen as the **addition** of a **small random variation to a point** in a multi-dimensional space.



Implementation

$$\begin{aligned} \text{Offspring}[1] &= \text{Parent}[1] + \text{sigma} \cdot \mathrm{R}_1 \\ &\vdots \\ \text{Offspring}[N] &= \text{Parent}[\mathrm{N}] + \text{sigma} \cdot \mathrm{R}_N \end{aligned}$$

where:

- *sigma* is a suitable positive constant and
- $\mathrm{R}_1, \ldots, \mathrm{R}_\mathrm{N}$ are (different) random numbers either <u>between -1.0 and 1.0</u> or <u>normal/gaussian</u> deviates
- In python:

```
import random

R = random.uniform(-1, 1)
R = random.gauss(0, 1)
```

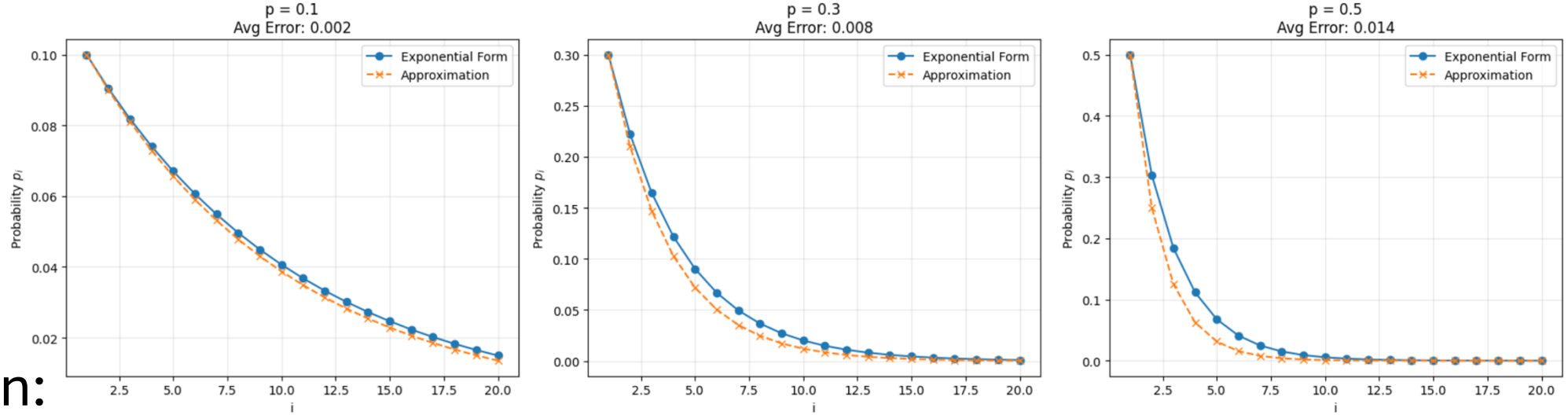
Rank selection

- Individuals are sorted (ranked) on the basis of their fitness, so $f_1 \geq f_2 \geq \ \dots \geq f_M$
- Then each individual is assigned a probability of being selected p_i taken from a given distribution, with the constraint

$$p_1 + p_2 + \dots + p_M = 1$$

Typical distribution: Exponential

- **Exponential**: $p_i = p \cdot e^{-p\cdot(i-1)}$
- For small p: $e^{-p} \cong (1-p)$ → $p_i = p \cdot (e^{-p})^{(i-1)} \cong p \cdot (1-p)^{(i-1)}$



- In Python:

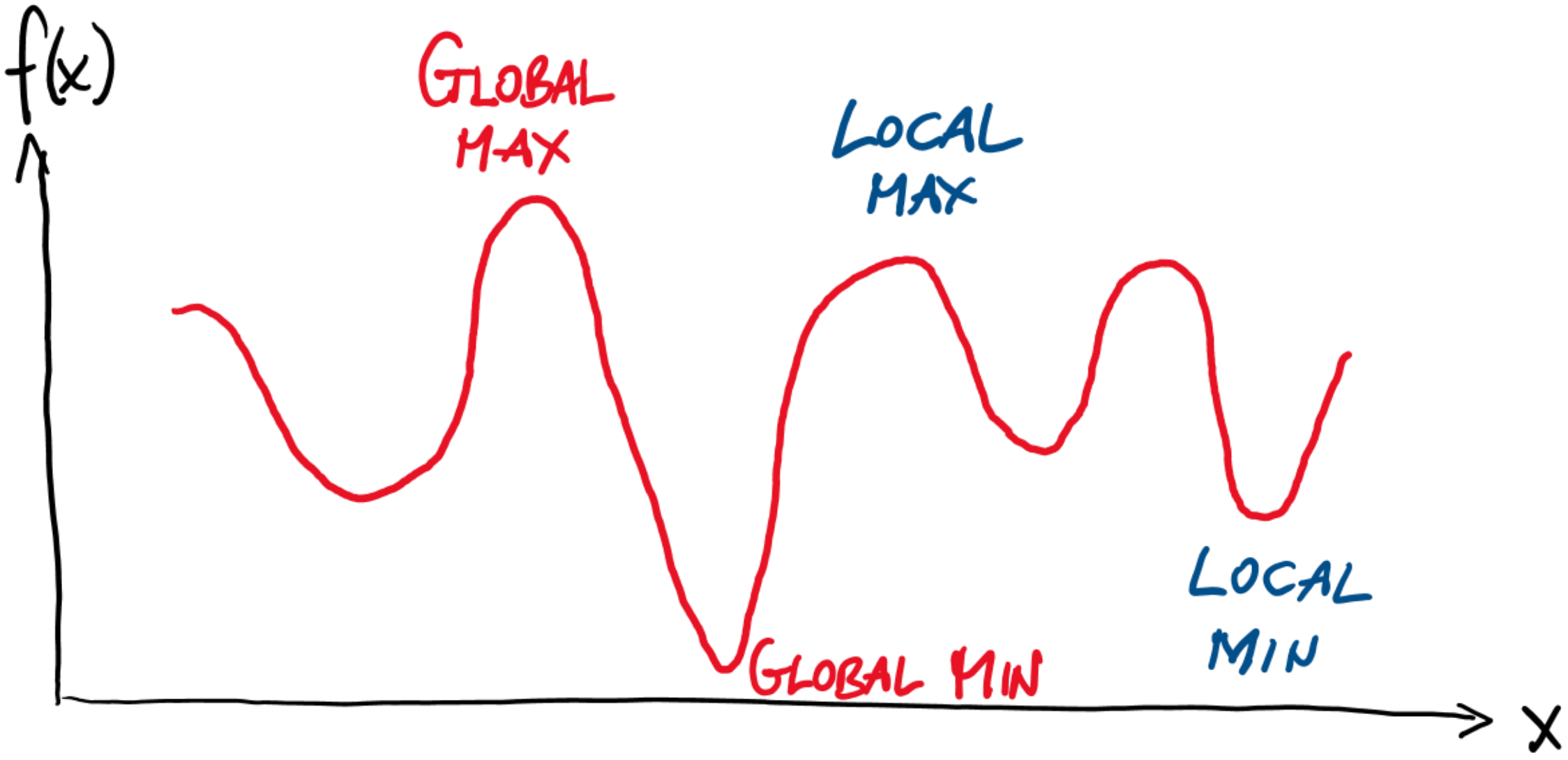
```
selected = 1
while True:
  if random number < p:
    return selected
  else:
    selected += 1
```

Rank Selection

- Advantages:
  - **No premature convergence**: no individual has N_i much bigger than 1
  - **No stagnation**: even at the end $N_1, N_2, \ldots$ are different. Explicit fitness not needed: only the ability to compare pairs of solutions is necessary.
- Disadvantages:
  - Reordering overhead
  - Little biological plausibility

Elitist selection

- At least a copy of the **best N individuals** of the population is always **passed to the new generation** (often $N = 1$)
- Advantage:
  - **Convergence is guaranteed**, i.e., if the global maximum is discovered, the GA converges to such maximum
- Disadvantage:
  - Risk of being trapped by **local maxima**


f(x)
GLOBAL MAX
LOCAL MAX
GLOBAL MIN
LOCAL MIN
X

Mapping Objective Functions into Fitness Functions (1)

- Case 1:
  - In many problems a quality measure $\boldsymbol{Q(s)}$ is known such that for all solutions $\boldsymbol{Q(s)} \geq \boldsymbol{0}$ and the objective is to find $\boldsymbol{s}^*$ for which $\boldsymbol{Q(s^*)} = \boldsymbol{max_s\, Q(s)}$.
- Solution:
  - We can use as a fitness measure

    $\boldsymbol{f(s)} = \boldsymbol{Q(s)}$

Mapping Objective Functions into Fitness Functions (2)

- Case 2:

  In some problems $\boldsymbol{Q}(\boldsymbol{s}) < \boldsymbol{0}$ for some solutions $\boldsymbol{s}$. So, FPS cannot be used ($f < 0$ gives negative selection probability!).

- Solution:

  Use an offset and a threshold:

  $$f(s) = Q(s) - C \qquad \text{if } Q(s) - C > 0$$
  $$f(s) = 0 \qquad \text{otherwise}$$

  where

  - C = minimum value $Q(s)$ can take (when known) or
  - C = minimum $Q(s)$ in the current generation (or last few gens)

Mapping Objective Functions into Fitness Functions (3)

- Case 3:
  - In some problems the natural measure of quality is actually a **cost** or an **error** $\boldsymbol{E(s)}$ and the problem is:

$$\text{find } \boldsymbol{s}^* \text{ such that } \boldsymbol{E(s^*)} = \boldsymbol{mins\ E(s)}.$$

- Solution:
  - Take a quality measure to be maximised
  - $\boldsymbol{Q(s)} = -\boldsymbol{E(s)}$ and go back to Case 1.
  - **If** $\boldsymbol{Q(s)} < \boldsymbol{0}$ for some **s** go to Case 2, instead.

Mapping Objective Functions into Fitness Functions (4)

- Case 4:
  - If, for all **s**, Q(s) is in the range $[\boldsymbol{Q_{min}}, \boldsymbol{Q_{max}}]$ with
    - $\boldsymbol{Q_{min}}$ much bigger than 0, but
    - $\boldsymbol{Q_{max}} - \boldsymbol{Q_{min}}$ much smaller than $\boldsymbol{Q_{min}}$.

    E.g., $[1000,1010]$
  - FPS can lead to stagnation even at a beginning of a run!
- Solution:
  - Go back to Case 2 and use $\boldsymbol{C} = \boldsymbol{Q_{min}}$ so f(**s**) is in the range $[\mathbf{0}, \boldsymbol{Qmax} - \boldsymbol{Q_{min}}]$

GAs for Constrained Optimisation

- Constrained optimisation means "**find the minimum of U in a predefined region of the space of possible solutions**".
- Usually this is formulated as:

  Minimise $\boldsymbol{U}(\boldsymbol{x})$ subject to the constraints:

  $\boldsymbol{g_i}(\boldsymbol{x}) \geq \mathbf{0}$ for $i = 1, \ldots, m$ (type 1, **inequality constraints**)

  $\boldsymbol{h_j}(\boldsymbol{x}) = \mathbf{0}$ for $j = 1, \ldots, n$ (type 2, **equality constraints**).

  (where **x** is a set of variables)

- Does it remind you of the Knapsack problem? (*combinatorial* constrained optimisation)

Example (1)

- Two variables (x = (x_1, x_2)), no equality constraints ($n = 0$) and one inequality constraint ($m = 1$):

$$\text{“}\textbf{Minimise } \boldsymbol{U(x_1, x_2) = x_1{}^2 + x_2{}^2}$$

$$\text{with } \boldsymbol{g(x_1, x_2) = x_1 * x_2 \geq 0}\text{"}$$

- Note that different kinds of constraints can be transformed into type 1 or type 2. For example, $x_1 + x_2 - 3 \leq 0$ is equivalent to $3 - x_1 - x_2 \geq 0$.

Penalty Method (1)

- **Constrained optimisation** problems can be transformed into **unconstrained optimisation** problems using the **penalty method**.
- Instead of minimising *U(x)*, minimize

$$U'(x) = U(x) + \boldsymbol{r} \cdot \boldsymbol{F_{Penalty}}\big(\boldsymbol{g_1}(\boldsymbol{x})\big) + \boldsymbol{r} \cdot \boldsymbol{F_{Penalty}}\big(\boldsymbol{g_2}(\boldsymbol{x})\big) + \cdots$$

where ***r*** is a penalty coefficient and $\boldsymbol{F_{Penalty}}$ is a penalty function.

Penalty Method (2)

- A typical penalty function is

$$\boldsymbol{F_{Penalty}}\big(g_i(x)\big) = g_i^2(x) \qquad \text{if } g_i(x) < 0$$
$$\boldsymbol{F_{Penalty}}\big(g_i(x)\big) = 0 \qquad \text{otherwise}$$

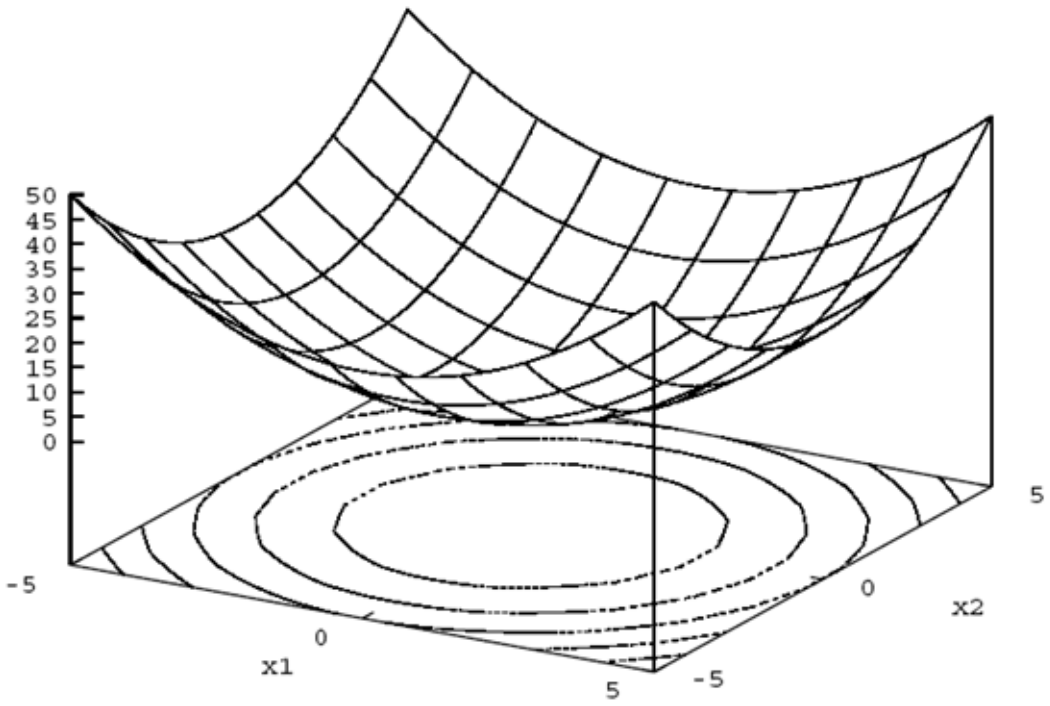
- For *equality constraints* we can use the same approach with penalty functions like $\boldsymbol{F_{Penalty}}\big(h_i(x)\big) = h_i^2(x)$

Penalty Method in Python

```
r = 10          # penalty coefficient

def U(x1, x2):
     return x1**2 + x2**2

def g(x1, x2):
     return 3 - x1 - x2

def penalty( constraint ):
     return constraint**2 if constraint < 0 else 0

def U_prime(x1, x2):
     return U(x1,x2) + r * penalty( g(x1, x2) )
```

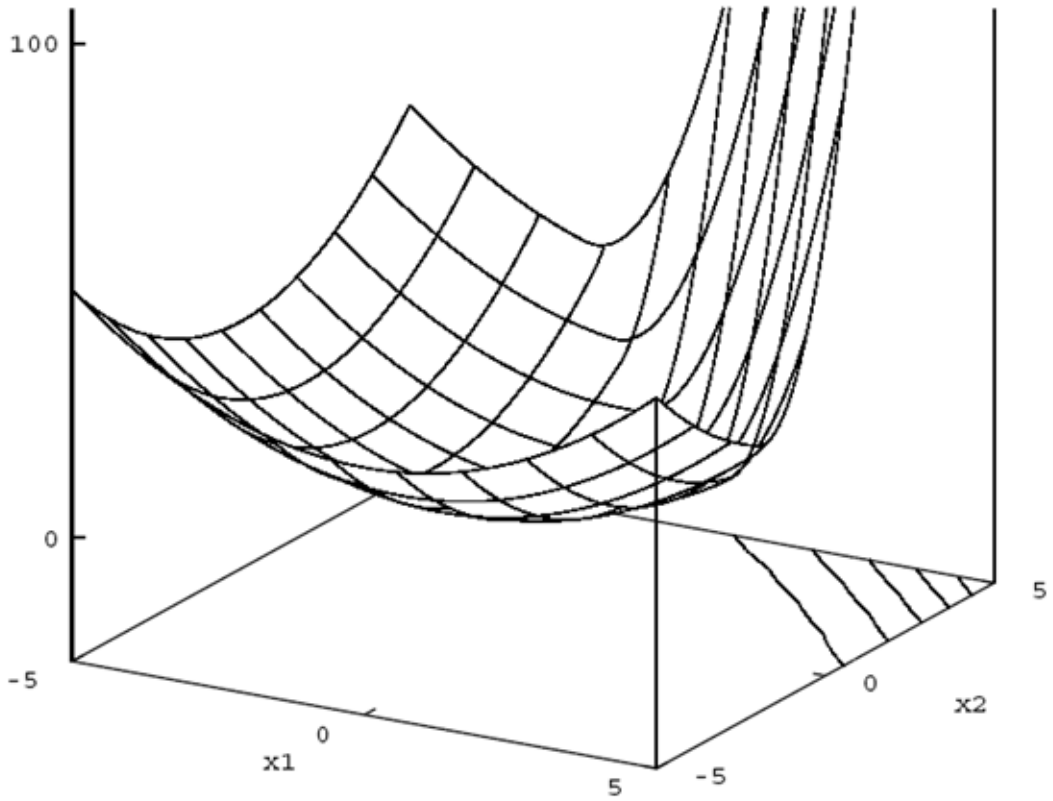
Example (2)

- Assume $U(x_1, x_2) = {x_1}^2 + {x_2}^2$ and $g(x_1, x_2) = 3 - x_1 - x_2 \geq 0$
- Before the penalty has been added, the objective function is



Example (3)

- After the penalty has been added ($r = 10$), the objective function is



Penalty Method (3)

- **Advantage**: no hard constraints on the values of x
- **Disadvantage**: some constraints can be “slightly” violated (e.g., by a good solution close to the border of the space of valid solutions).
- For most problems of combinatorial optimisation this is *not acceptable*, as the constraints enforce the syntactic correctness of the solutions
- **Alternatives**:
  - special representations and operators
  - repair slightly invalid solutions

What did we learn today? (1)

- We have learned that **unconstrained function optimisation** (**UFO**) is about finding the values for the parameters x that minimise or maximise a given objective function U(x), and that the difference between UFO and **unconstrained combinatorial optimisation** is only in the nature of the variables: continuous vs. discrete.
- We have learned how to develop **fitness** functions for various problems and saw that solving new problems "**just**" requires definition of representation and fitness function.

What did we learn today? (2)

- We have seen how **real-valued GAs** operate on arrays of real numbers better than binary GAs but require **new crossover and mutation operators**.
- We have introduced **rank selection** and found that it can implemented easily
- We have seen the benefits and risks of **elitism**
- We have understood how to **transform objective functions** (U) **into fitness functions** (f)
- We have seen how **constrained optimisation** can be transformed into unconstrained optimisation using **penalties**

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Next time:
Evolution strategies

University of Essex
School of
Computer Science and
Electronic Engineering